

文档与视图

党宇

dangyu@Nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

目录

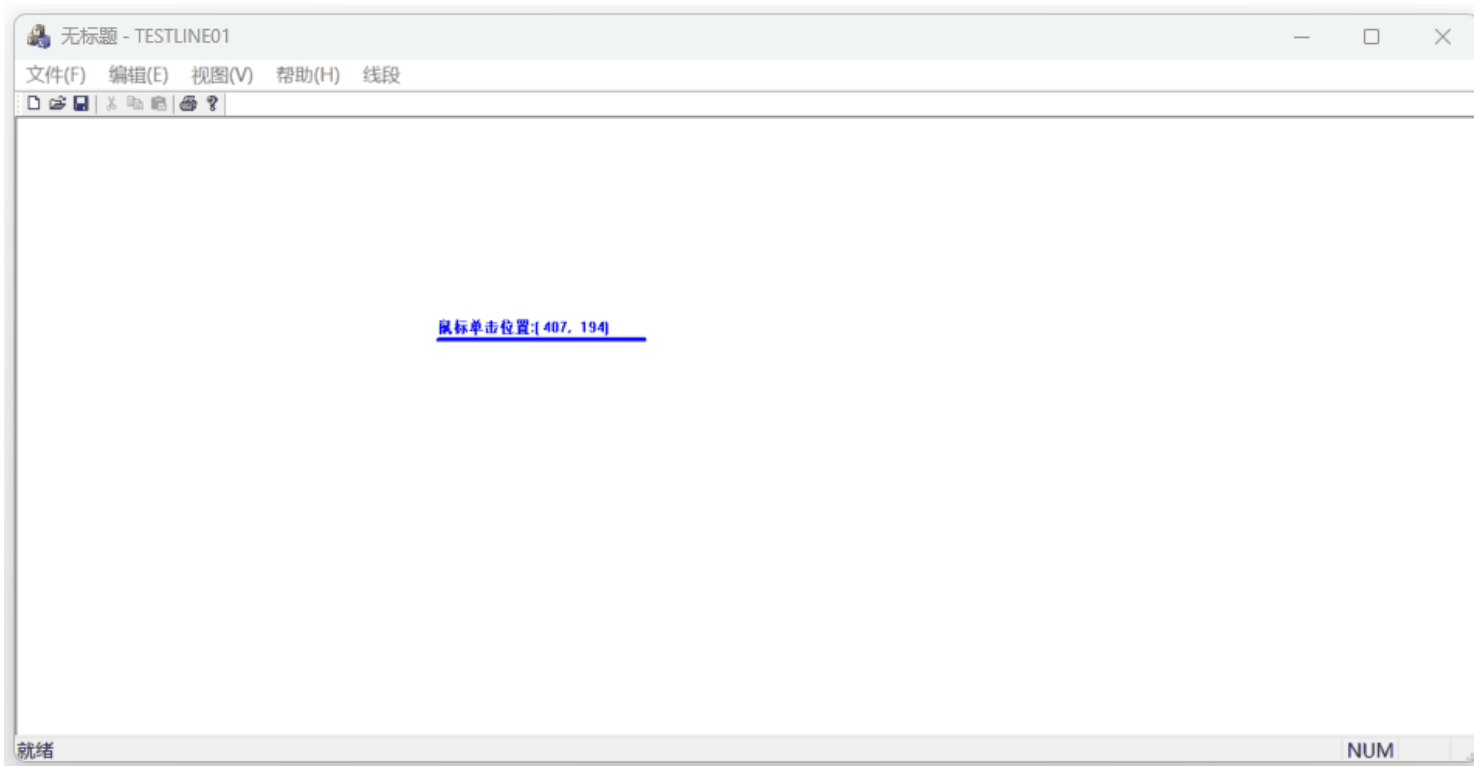
- 文档与视图
- 资源的应用
- WM_TIMER消息
- 综合实例
 - 绘制曲线动画
 - 位图资源的应用
 - 文档/视图的专题练习

专题练习

- 1. 单击鼠标左键，在视图中显示点击位置，并进行保存。（知识点：文档/视图）
- 2. 创建一个包含有“黑色” “红色” “绿色” “蓝色” 四个选项的应用程序，能够对显示的线条颜色进行改变。（知识点：资源的应用，绘图颜色）
- 3. 在习题2基础上，在菜单栏添加开始和停止键。按下开始键，可以利用计数器实现线条的闪烁和颜色变化。按下停止键，可以停留到当前颜色。可以保存并打开查看线条的位置和颜色，并能继续闪烁。（知识点：计时器）
- 4. 创建一个单文档应用程序，连续左键单击三个点，连线绘成三角形。添加颜色按钮菜单，能够实现颜色改变。能够保存并打开三角形形状和颜色。

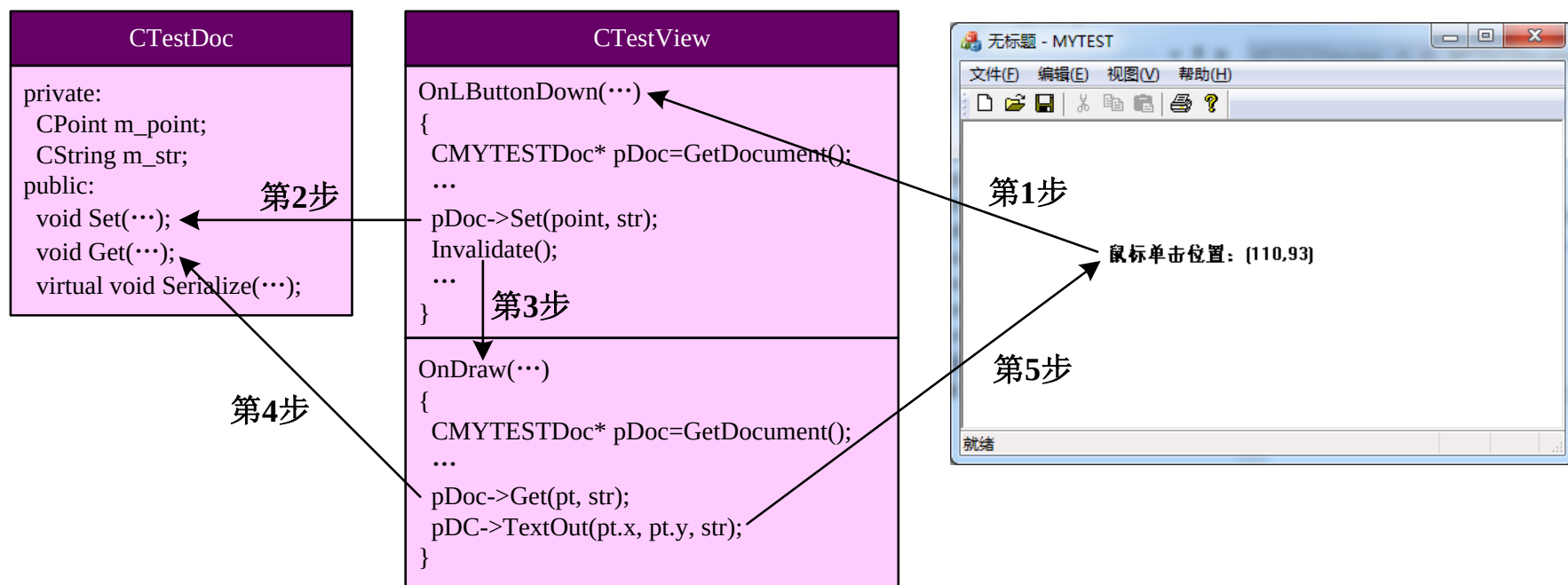
专题练习-习题1

- 1. 单击鼠标左键，在视图中显示点击位置，并进行保存。（知识点：文档/视图）



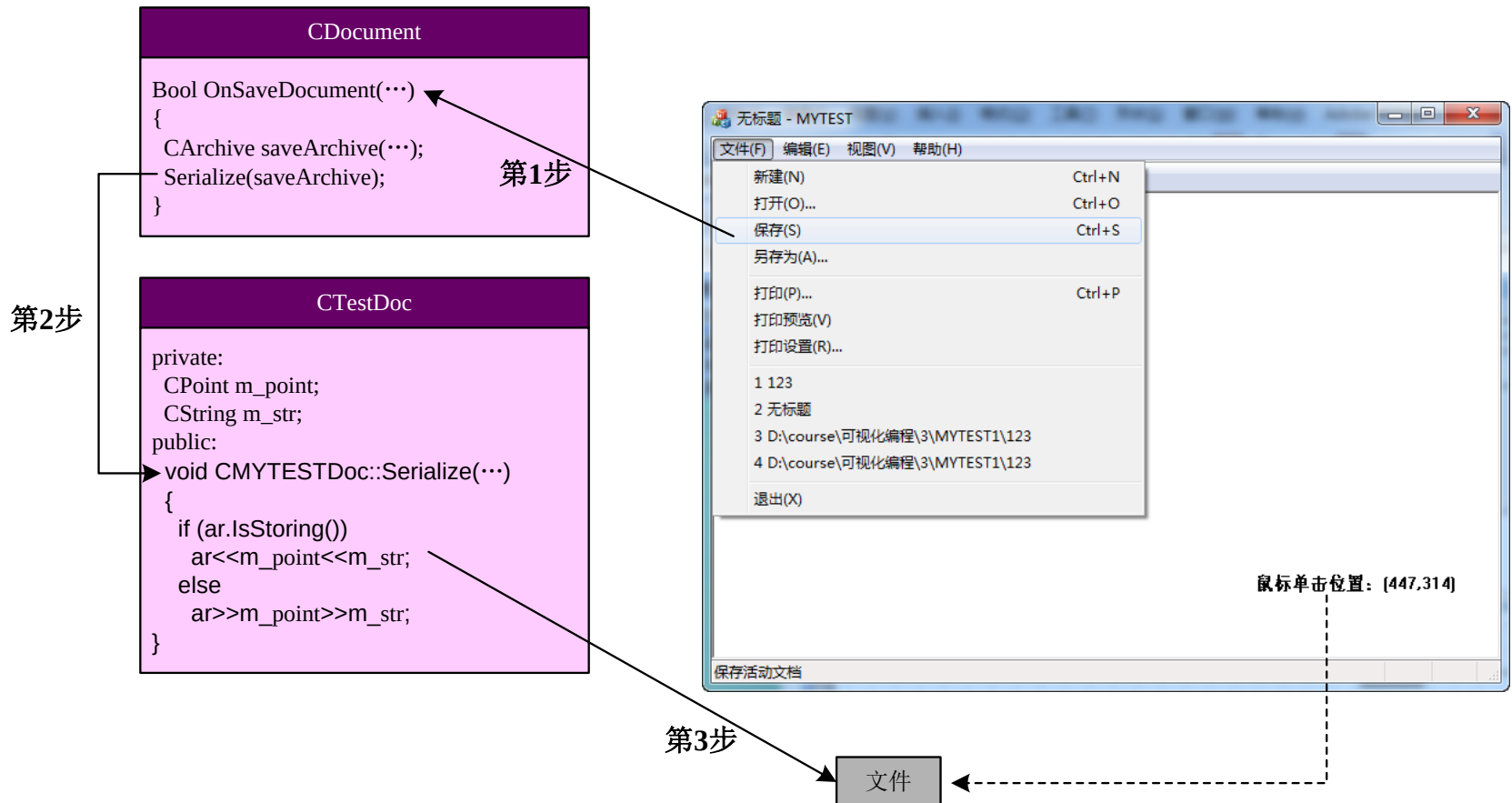
专题练习-习题1分析

➤ 鼠标点击过程



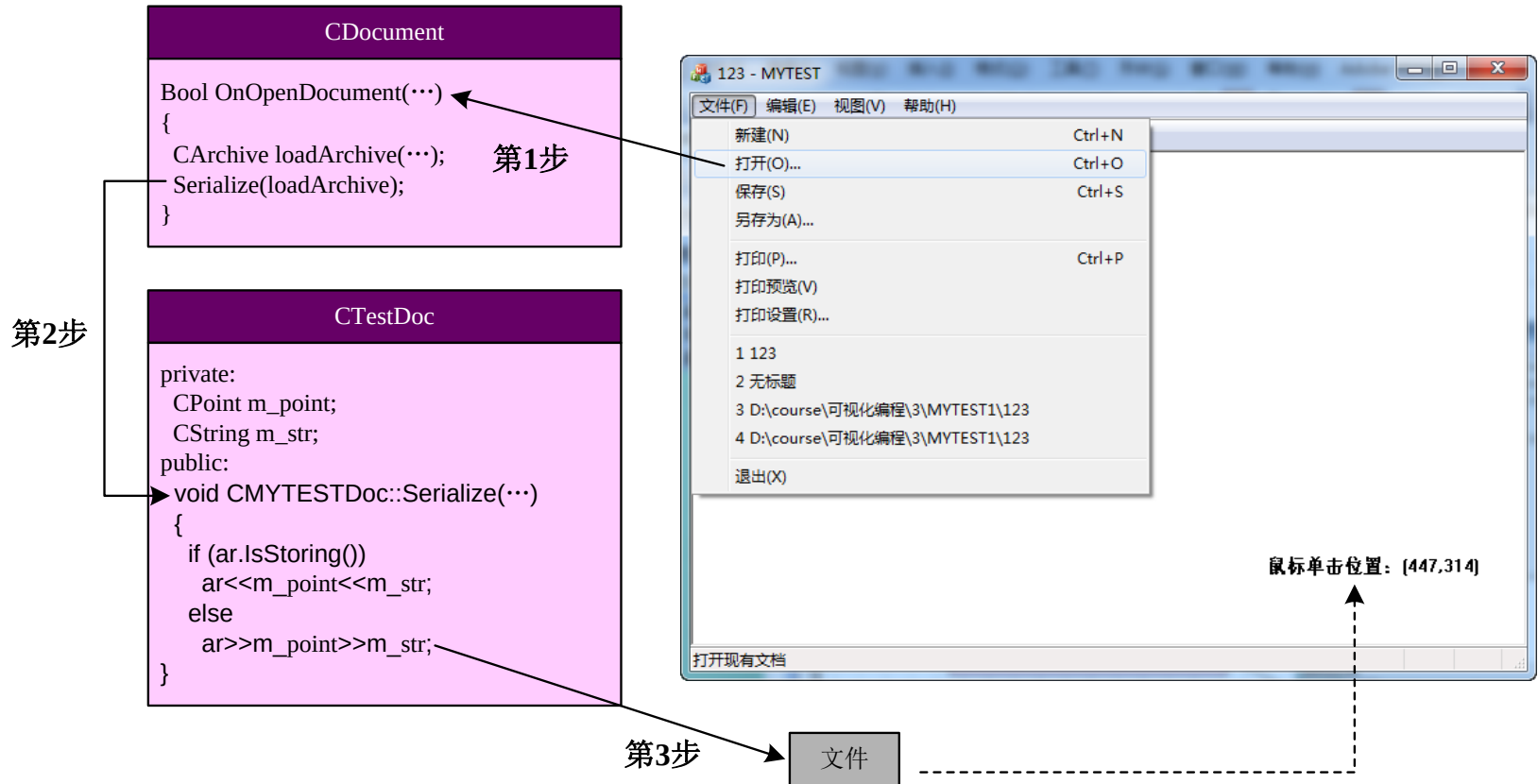
专题练习-习题1分析

➤ 文件保存过程



专题练习-习题1分析

➤ 文件读取过程



专题练习-习题2

- 2.创建一个包含有“黑色” “红色” “绿色” “蓝色” 四个选项的应用程序，能够对显示的线条颜色进行改变。（知识点：资源的应用，绘图颜色）

专题练习-习题2分析

1. 添加成员变量和相应函数

在CMYTESTDoc类定义中(MYTESTDoc.h)添加

private:

CString m_str;

CPoint m_point;

int m_nColorIndex; // 代表颜色

public:

void Set(CPoint point, CString string);

void Get(CPoint& point, CString& string);

void SetColor(int index);

int GetColor();

添加蓝色
代码

专题练习-习题2分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
CTESTLINE01Doc::CTESTLINE01Doc() noexcept  
{  
    // TODO: 在此添加一次性构造代码  
    m_str.Empty();  
    m_point = CPoint(0, 0);  
    m_nColorIndex = 0; // 默认黑色  
}
```

添加蓝色
代码

专题练习-习题2分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CMYTESTDoc::Set(CPoint point, CString string)
{   m_point = point;
    m_str = string;}

void CMYTESTDoc ::Get(CPoint& point, CString& string)
{   point = m_point;
    string = m_str;}

void CMYTESTDoc ::SetColor(int index)
{   m_nColorIndex = index; }

int CMYTESTDoc ::GetColor()
{   return m_nColorIndex; }
```

添加蓝色
代码

专题练习-习题2分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CMYTESTDoc::Serialize(CArchive& ar) {  
    if(ar.IsStoring())        //写操作  
        ar<<m_point<<m_str<<m_nColorIndex;  
    else                        //读操作  
        ar>>m_point>>m_str>>m_nColorIndex;  
}
```

添加蓝色
代码

专题练习-习题2分析

1. 添加成员变量和相应函数

在CMYTESTView类定义中(MYTESTView.h)添加

private:

```
COLORREF m_nColors[4];    // 颜色值
```

MYTESTView.cpp

```
CMYTESTView::CMYTESTView()
```

```
{
```

```
    m_nColors[0] = RGB(0,0,0);           // 黑色
```

```
    m_nColors[1] = RGB(255,0,0);         // 红色
```

```
    m_nColors[2] = RGB(0,255,0);         // 绿色
```

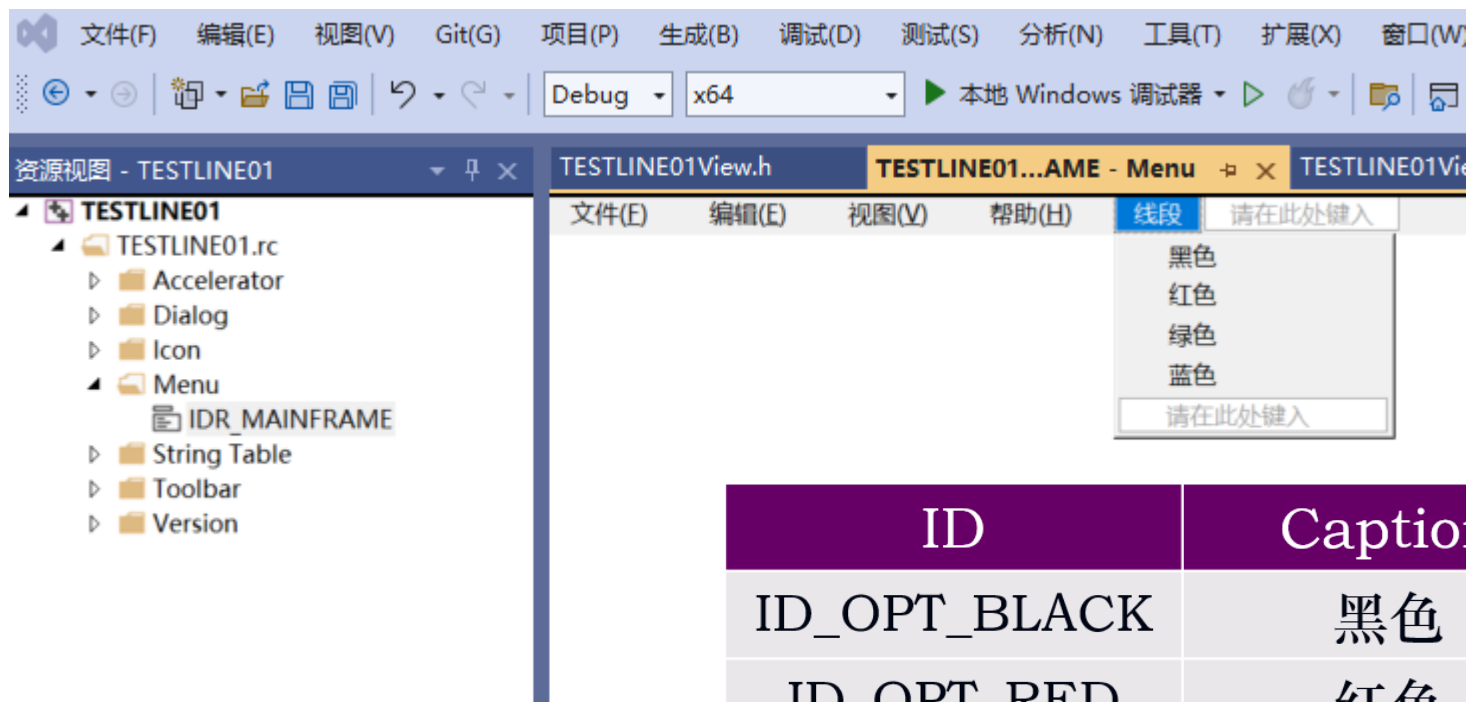
```
    m_nColors[3] = RGB(0,0,255);         // 蓝色
```

```
}
```

添加蓝色
代码

专题练习-习题2分析

2. 添加菜单资源



ID	Caption
ID_OPT_BLACK	黑色
ID_OPT_RED	红色
ID_OPT_GREEN	绿色
ID_OPT_BLUE	蓝色

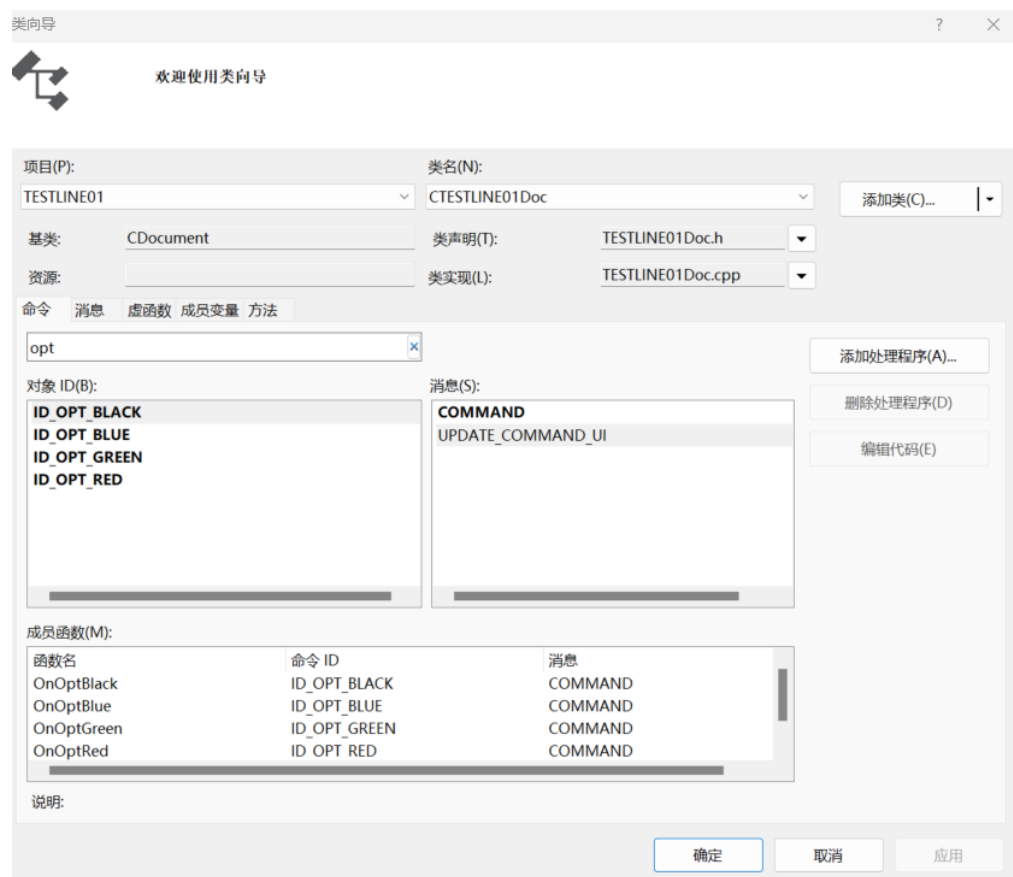
专题练习-习题2分析

3. 响应COMMAND消息

用户单击菜单项时产生COMMAND消息

➤ 在Doc类中响应消息

- ID_OPT_BLACK
- ID_OPT_RED
- ID_OPT_GREEN
- ID_OPT_BLUE



专题练习-习题2分析

3. 响应COMMAND消息

MYTESTDoc.cpp

```
void CMYTESTDoc::OnOptBlack()
{
    m_nColorIndex = 0;
    UpdateAllViews(NULL);
} // 其他颜色相同操作，替换相应的m_nColorIndex
```

专题练习-习题2分析

4. 响应鼠标左键的消息，确定线段的左端点位置

```
OnLButtonDown(UINT nFlags, CPoint point) {  
  
    CTESTDoc* pDoc = GetDocument();  
    CString str;  
    str.Format(_T("鼠标单击位置:(% d, % d)"), point.x, point.y);  
    pDoc->Set(point, str);  
    Invalidate();  
  
    CView::OnLButtonDown(nFlags, point);  
}
```



专题练习-习题2分析

5. 绘制线段

MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC) {  
    ... CPoint pt;  
    CString str;  
    pDoc->Get(pt, str);  
    pDC->SetTextColor(m_nColors[pDoc->GetColor()]);  
    pDC->TextOutW(pt.x, pt.y, str);  
    // 绘制横线  
    CPen NewPen(PS_SOLID, 4, m_nColors[pDoc->GetColor()]);  
    pDC->SelectObject(&NewPen);  
    pDC->MoveTo(pt.x, pt.y + 20);  
    pDC->LineTo(pt.x + 200, pt.y+20);}
```

添加蓝色
代码

专题练习-习题3

- 3. 在习题2基础上，在菜单栏添加开始和停止键。按下开始键，可以利用计数器实现线条的闪烁和颜色变化。按下停止键，可以停留到当前颜色。可以保存并打开查看线条的位置和颜色，并能继续闪烁。（知识点：计时器）

专题练习-习题3分析

1. 添加成员变量和相应函数

在CMYTESTDoc类定义中(MYTESTDoc.h)添加

private:

CString m_str;

CPoint m_point;

int m_nColorIndex; // 代表颜色

public:

void Set(CPoint point, CString string);

void Get(CPoint& point, CString& string);

void SetColor(int index);

int GetColor();

添加蓝色
代码

(Doc类的头文件和源文件的代码与习题2相同)

专题练习-习题3分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
CTESTLINE01Doc::CTESTLINE01Doc() noexcept  
{  
    // TODO: 在此添加一次性构造代码  
    m_str.Empty();  
    m_point = CPoint(0, 0);  
    m_nColorIndex = 0; // 默认黑色  
}
```

添加蓝色
代码

专题练习-习题3分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CTESTLINE01Doc::Set(CPoint point, CString string)
{   m_point = point;
    m_str = string;}

void CTESTLINE01Doc::Get(CPoint& point, CString& string)
{   point = m_point;
    string = m_str;}

void CMYTESTDoc::SetColor(int index)
{   m_nColorIndex = index; }
int CMYTESTDoc::GetColor()
{   return m_nColorIndex; }
```

添加蓝色
代码

专题练习-习题3分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CMYTESTDoc::Serialize(CArchive& ar) {  
    if(ar.IsStoring())        //写操作  
        ar<<m_point<<m_str<<m_nColorIndex;  
    else                        //读操作  
        ar>>m_point>>m_str>>m_nColorIndex;  
}
```

添加蓝色
代码

专题练习-习题3分析

1. 添加成员变量和相应函数

在CMYTESTView类定义中(MYTESTView.h)添加

private:

```
COLORREF m_nColors[4];    // 颜色值
```

```
int m_num; // 绘制数目
```

MYTESTView.cpp

```
CMYTESTView::CMYTESTView()
```

```
{
```

```
    m_nColors[0] = RGB(0,0,0);           // 黑色
```

```
    m_nColors[1] = RGB(255,0,0);         // 红色
```

```
    m_nColors[2] = RGB(0,255,0);         // 绿色
```

```
    m_nColors[3] = RGB(0,0,255);         // 蓝色
```

```
}
```

添加蓝色
代码

专题练习-习题3分析

2. 添加菜单资源



ID	Caption
ID_OPT_BLACK	黑色
ID_OPT_RED	红色
ID_OPT_GREEN	绿色
ID_OPT_BLUE	蓝色

ID	Caption
ID_OPT_START	开始
ID_OPT_STOP	停止

专题练习-习题3分析

3. 响应COMMAND消息

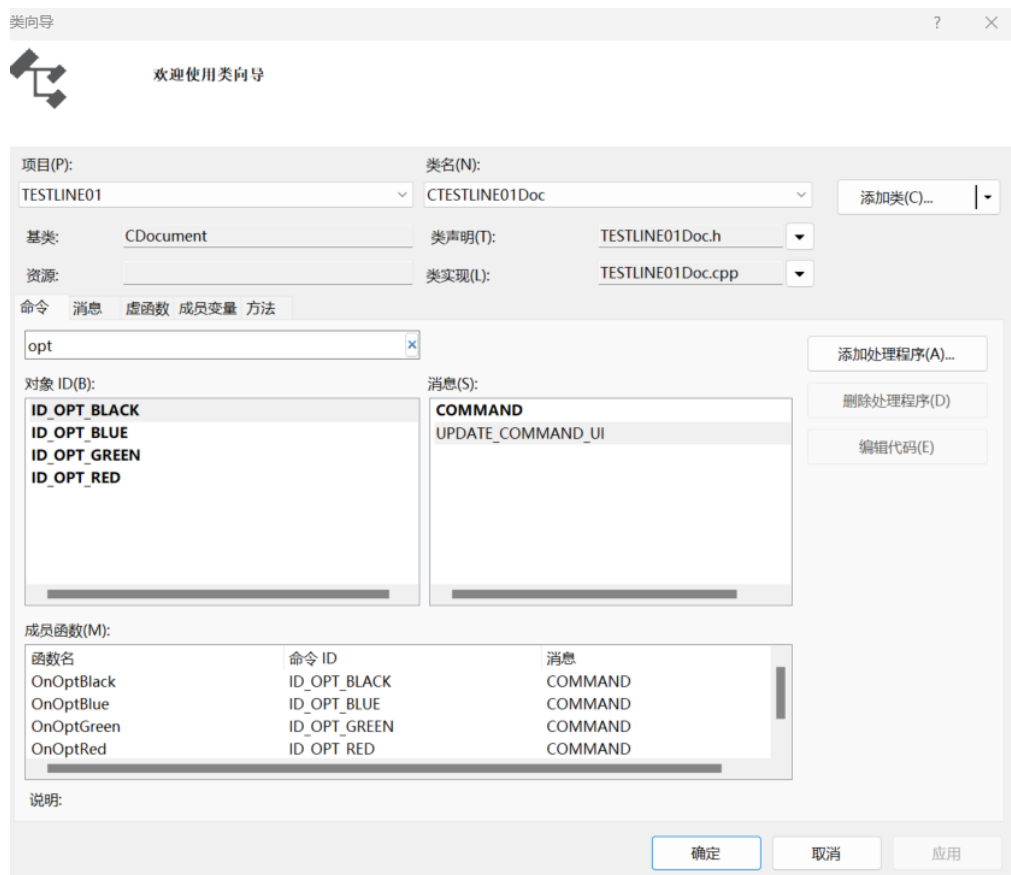
用户单击菜单项时产生COMMAND消息

➤ 在Doc类中响应消息

- ID_OPT_BLACK
- ID_OPT_RED
- ID_OPT_GREEN
- ID_OPT_BLUE

➤ 在View类中响应消息

- ID_OPT_START
- ID_OPT_STOP



专题练习-习题3分析

3. 响应颜色COMMAND消息

MYTESTDoc.cpp

```
void CMYTESTDoc::OnOptBlack()
{
    m_nColorIndex = 0;
    UpdateAllViews(NULL);
} // 其他颜色相同操作，替换响应的m_nColorIndex
```

专题练习-习题3分析

4. 响应开始和停止COMMAND消息

MYTESTView.cpp

```
void CMYTESTView::OnOptStart ()  
{   SetTimer(1, 500, NULL);}
```

```
void CMYTESTView ::OnOptStop()  
{   // TODO: 在此添加命令处理程序代码  
    KillTimer(1);  
    MYTESTVDoc* pDoc = GetDocument();  
    pDoc->SetColor((pDoc->GetColor() + m_num-1) % 4);  
}
```



专题练习-习题3分析

5. onTimer()函数

MYTESTView.cpp

```
void CTESTLINE01View::OnTimer(UINT_PTR nIDEvent)
{
    // TODO: 在此添加消息处理程序代码和/或调用默认值
    if (nIDEvent == 1) {
        CClientDC dc(this);
        MYTESTDoc * pDoc = GetDocument();
        CPoint pt; CString str;

        pDoc->Get(pt, str);
        dc.SetTextColor(m_nColors[(pDoc->GetColor() + m_num) % 4]);
        dc.TextOutW(pt.x, pt.y, str);
    }
}
```

//……(接下页)……

专题练习-习题3分析

5. onTimer()函数

```
//.....(接上页).....
```

```
    CPen NewPen(PS_SOLID, 4, m_nColors[(pDoc->GetColor()+  
m_num) % 4]);
```

```
    dc.SelectObject(&NewPen);
```

```
    dc.MoveTo(pt.x, pt.y + 20);
```

```
    dc.LineTo(pt.x + 200, pt.y + 20);
```

```
    CString strNUM;
```

```
    strNUM.Format(_T("变化次数: %d"), m_num);
```

```
    dc.TextOutW(pt.x, pt.y+40, strNUM);
```

```
    m_num++;}
```

```
    CView::OnTimer(nIDEvent);
```

```
}
```

专题练习-习题3分析

6. 响应鼠标左键的消息，确定线段的左端点位置

```
OnLButtonDown(UINT nFlags, CPoint point) {  
  
    CTESTDoc* pDoc = GetDocument();  
    CString str;  
    str.Format(_T("鼠标单击位置:(% d, % d)"), point.x, point.y);  
    pDoc->Set(point, str);  
    Invalidate();  
  
    CView::OnLButtonDown(nFlags, point);  
}
```



专题练习-习题3分析

7. 绘制线段

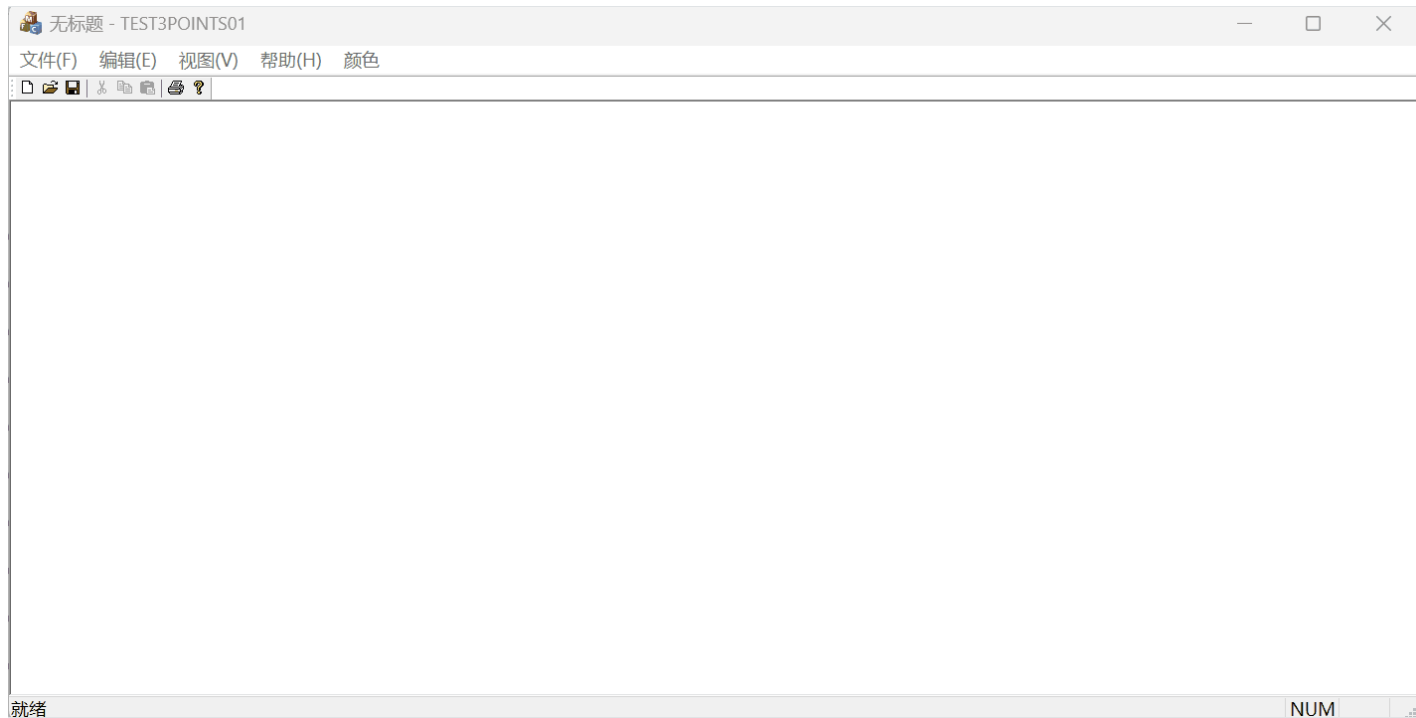
MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC) {  
    ... CPoint pt;  
    CString str;  
    pDoc->Get(pt, str);  
    pDC->SetTextColor(m_nColors[(pDoc->GetColor() + m_num) % 4]);  
    pDC->TextOutW(pt.x, pt.y, str);  
    // 绘制横线  
    CPen NewPen(PS_SOLID, 4, m_nColors[(pDoc->GetColor() + m_num) % 4]);  
    pDC->SelectObject(&NewPen);  
    pDC->MoveTo(pt.x, pt.y + 20);  
    pDC->LineTo(pt.x + 200, pt.y+20);  
    CString strNUM;  
    strNUM.Format(_T("变化次数: %d"), m_num);  
    pDC->TextOutW(pt.x, pt.y + 40, strNUM);}
```

添加蓝色
代码

专题练习-习题4

- 4.创建一个单文档应用程序，连续左键单击三个点，连线绘成三角形。添加颜色按钮菜单，能够实现颜色改变。能够保存并打开三角形形状和颜色。



专题练习-习题4分析

1. 添加成员变量和相应函数

在CMYTESTDoc类定义中(MYTESTDoc.h)添加

public:

```
CPoint m_npoint[3];
```

```
int m_ncount;
```

```
int m_nColorIndex;
```

public:

```
void SetColor(int index);
```

```
int GetColor();
```

添加蓝色
代码

专题练习-习题4分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
CTESTLINE01Doc::CTESTLINE01Doc() noexcept
{
    // TODO: 在此添加一次性构造代码
    m_ncount = 0;
    m_nColorIndex = 0; //默认黑色
    m_npoint[3] = NULL;
}
```

添加蓝色
代码

专题练习-习题4分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void MYTESTDoc ::SetColor(int index)
{   m_nColorIndex = index; }

int MYTESTDoc ::GetColor()
{   return m_nColorIndex; }
```

添加蓝色
代码

专题练习-习题4分析

1. 添加成员变量和相应函数

MYTESTDoc.cpp

```
void CMYTESTDoc::Serialize(CArchive& ar) {  
    if(ar.IsStoring())        //写操作  
        m_ncount = 3;  
    ar << m_npoint[0] << m_npoint[1] << m_npoint[2] <<  
    m_ncount << m_nColorIndex;  
    else                        //读操作  
        m_ncount = 3;  
    ar >> m_npoint[0] >> m_npoint[1] >> m_npoint[2] >> m_ncount  
    >> m_nColorIndex;  
}
```

添加蓝色
代码

专题练习-习题4分析

1. 添加成员变量和相应函数

在CMYTESTView类定义中(MYTESTView.h)添加

private:

```
COLORREF m_nColors[4];    // 颜色值
```

MYTESTView.cpp

```
CMYTESTView::CMYTESTView()
```

```
{
```

```
    m_nColors[0] = RGB(0,0,0);           // 黑色
```

```
    m_nColors[1] = RGB(255,0,0);         // 红色
```

```
    m_nColors[2] = RGB(0,255,0);         // 绿色
```

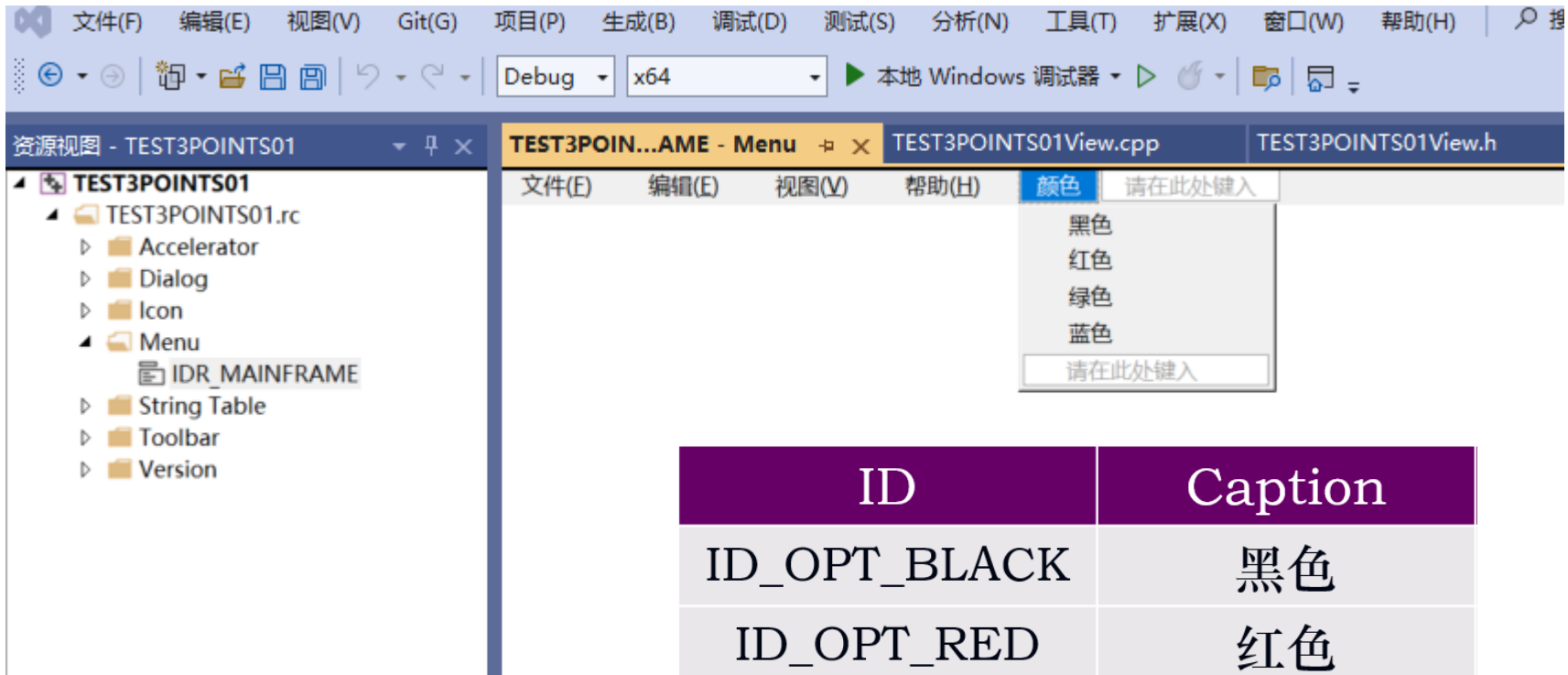
```
    m_nColors[3] = RGB(0,0,255);         // 蓝色
```

```
}
```

添加蓝色
代码

专题练习-习题4分析

2. 添加菜单资源



ID	Caption
ID_OPT_BLACK	黑色
ID_OPT_RED	红色
ID_OPT_GREEN	绿色
ID_OPT_BLUE	蓝色

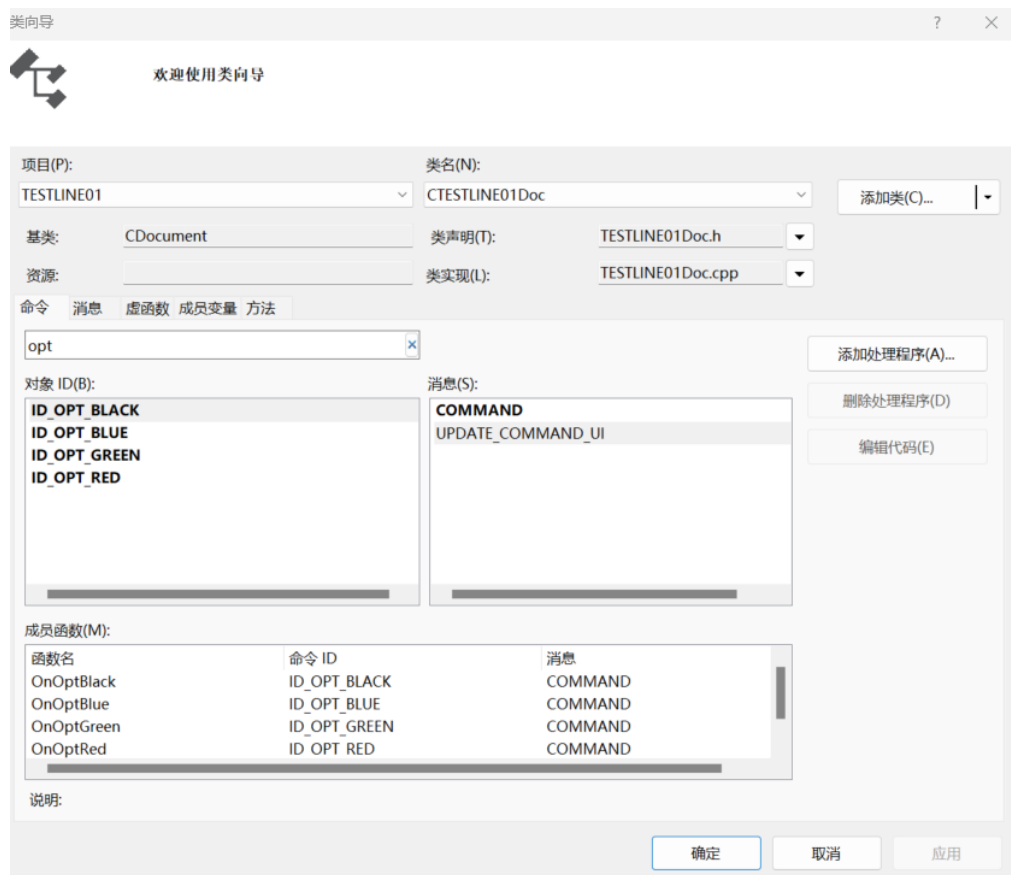
专题练习-习题4分析

3. 响应COMMAND消息

用户单击菜单项时产生COMMAND消息

➤ 在Doc类中响应消息

- ID_OPT_BLACK
- ID_OPT_RED
- ID_OPT_GREEN
- ID_OPT_BLUE



专题练习-习题4分析

3. 响应COMMAND消息

MYTESTDoc.cpp

```
void CMYTESTDoc::OnOptBlack()
{
    m_nColorIndex = 0;
    UpdateAllViews(NULL);
} // 其他颜色相同操作，替换响应的m_nColorIndex
```

专题练习-习题4分析

4. 响应鼠标左键的消息，确定线段的左端点位置

```
OnLButtonDown(UINT nFlags, CPoint point) {  
  
    CTESTDoc* pDoc = GetDocument();  
  
    pDoc->m_npoint[pDoc->m_ncount++] = point ;  
  
    Invalidate();  
    CView::OnLButtonDown(nFlags, point);  
}
```

专题练习-习题4分析

5. 绘制线段

MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC) {  
    ...  
    if (pDoc->m_ncount == 3)  
    {  
        for (int i = 0; i < 3; i++)  
        {  
            CPoint pt;  
            pt = pDoc->m_npoint[i];  
            CString str;  
            str.Format(_T("鼠标单击位置: (%d,%d)"), pt.x, pt.y);  
            pDC->SetTextColor(m_nColors[pDoc->GetColor()]);  
            pDC->TextOut(pt.x, pt.y, str); //其他点类似  
        } //...接下页...
```

添加蓝色
代码



专题练习-习题4分析

5. 绘制三角形

添加蓝色
代码

MYTESTView.cpp

```
void CMYTESTView::OnDraw(CDC* pDC) {  
    //...接上页...  
    CPen NewPen(PS_SOLID, 5, m_nColors[pDoc->GetColor()]);  
    CPen* pOldPen;  
    pOldPen = pDC->SelectObject(&NewPen);  
    //绘制三角形  
    pDC->MoveTo(pDoc->m_npoint[0].x, pDoc->m_npoint[0].y);  
    pDC->LineTo(pDoc->m_npoint[1].x, pDoc->m_npoint[1].y);  
    pDC->MoveTo(pDoc->m_npoint[1].x, pDoc->m_npoint[1].y);  
    pDC->LineTo(pDoc->m_npoint[2].x, pDoc->m_npoint[2].y);  
    pDC->MoveTo(pDoc->m_npoint[2].x, pDoc->m_npoint[2].y);  
    pDC->LineTo(pDoc->m_npoint[0].x, pDoc->m_npoint[0].y);  
    pDoc->m_ncount = 0;  
}
```

练习题

- 1.分析本节课的专题练习，练习独立写出代码。
- 2.练习综合实例的例子
- 2.阅读教科书第10章和第11章